

An Enforcement Boundary for LLM Agents Operating on Spreadsheet Artifacts

the xlq authors

2026-07-03

Contents

1	An Enforcement Boundary for LLM Agents Operating on Spreadsheet Artifacts	1
1.1	Abstract	1
1.2	1. Introduction	1
1.3	2. The Fidelity Gap	3
1.4	3. A Format-Aware Enforcement Boundary	4
	1.4.1 Read-only by construction	5
	1.4.2 A privacy-safe structural census	5
	1.4.3 Coverage honesty	6
	1.4.4 The semantic diff and the specified write path	6
1.5	4. Coverage Honesty: Three Numbers, Not One	7
1.6	5. Documentation-Arbitered Differential Validation	9
	1.6.1 The disagreement matrix (the result)	9
	1.6.2 Bugs in both engines	10
	1.6.3 Concordance is not accuracy	10
	1.6.4 How fault was assigned, and how far it can be trusted	10
	1.6.5 What is, and is not, new here	11
1.7	6. AXLE-bench: Measuring the Omitted Dimension	11
	1.7.1 The five axes	12
	1.7.2 Instrument validation as a co-headline	13
	1.7.3 Adoption cost and known floors	13
	1.7.4 The comparison matrix	13
	1.7.5 Methodology caveats, stated straight	13
	1.7.6 The claim this suite supports	14
1.8	7. Results	14
1.9	8. Related Work: What We Did Not Invent	15
1.10	9. Discussion	17
	1.10.1 Is a boundary that only reads, and only <i>predicts</i> writes, worth building?	17
	1.10.2 When the substrate moved faster than the plan	17
	1.10.3 Load-bearing assumptions, stated plainly so they can be challenged	18
	1.10.4 Two primitives we hope outlast this system	18
	1.10.5 The door Pista left open	18
1.11	10. Limitations	19
1.12	11. Conclusion	19
1.13	References	20

1 An Enforcement Boundary for LLM Agents Operating on Spreadsheet Artifacts

Format-Aware Safety, Coverage Honesty, and Differential Validation

Authors omitted for review.

1.1 Abstract

When an LLM agent edits a spreadsheet, its decisions do not sit in a separate log — they become the cells the file's owner is accountable for. Yet the substrate agents use today silently corrupts the file: a single re-save through the standard Python library strips 101,961 cached formula values from a five-workbook corpus, and no existing agent benchmark measures the damage. We present `xlq`, a format-aware enforcement boundary that treats the workbook as an artifact to be preserved rather than a string to be rewritten. `xlq` is read-only by construction: it exposes a privacy-safe structural census so the agent reads structure, not content (up to $7,246.5\times$ smaller than a full-cell dump); it reports coverage as three honest numbers rather than one — 522 functions recognized, 505 locally evaluable, 17 returning the exact error literal Excel returns. Enforcement today is structural: `xlq` v0.1 exposes no write path at all, so an agent driving it cannot corrupt the file through it — writes are prevented by absence, not gated by a check. The mediated write layer that would let an agent *edit* safely — typed patches, dry-run semantic prediction, hash-chained receipts — is specified and ships in v0.2, not measured here. To earn trust in the engine rather than assume it, we built a differential oracle over 1,659 cases across 492 functions, using LibreOffice as a reference and Microsoft's documentation as arbiter; it validated the engine and surfaced real bugs in both it and LibreOffice. We package the evaluation as AXLE-bench, whose fidelity axis measures the dimension current benchmarks omit. All artifacts are reproducible.

1.2 1. Introduction

Anthropic's own Claude Code shipped a spreadsheet skill that quietly destroyed the files it edited (issue #22044): asked to change a few cells in a financial model, the agent completed the requested edit and reported success, but the workbook it saved had lost its charts, pivot tables, and macros — stripped by `openpyxl`, the standard Python library the skill used to write the file back to disk. The loss was invisible at exactly the moment it mattered: the agent's own report, the diff a reviewer would skim, and the cells the task touched all looked correct. When an LLM agent edits a spreadsheet, its decisions do not sit in a separate log — they become the cells the file's owner is accountable for. Yet the substrate agents use today silently corrupts the file: a single re-save through the standard Python library strips 101,961 cached formula values from a five-workbook corpus, and no existing agent benchmark measures the damage. Those values are recoverable in principle — the formulas that produce them survive — but only by a tool that recomputes; every consumer that trusts the stored cache, including the agent's own later reads and the person who opens the file, sees them gone. One substrate over, LibreOffice keeps the formulas but rewrites 90,448 cached results to fifteen significant digits, and both tools rewrite essentially every part of the OOXML container, so byte-identity carries no integrity signal. Across every substrate an agent might use, then, the file's stored answers cannot be trusted, and deciding whether an edit preserved a workbook means recomputing it rather than comparing bytes (Section 2).

This corruption is invisible to the field's instruments by convention, not by oversight. Every benchmark built to measure LLM agents on spreadsheets — SheetCopilot [Li et al. 2023], SheetAgent [SheetAgent 2025], SpreadsheetBench [SpreadsheetBench 2024] and its 2026 successor [SpreadsheetBench 2026], and Finch [Finch 2025] — scores an agent by comparing the cells the task was supposed to change against a reference answer. What happens to the cells, formulas, caches, and charts the task was *not* supposed to touch lies outside the scored region; SheetCopilot's checker skips unflagged properties by design. This is a defensible way to measure task competence, but it is a convention, not a law of the domain, and it makes file corruption unobservable: an agent that produces the right answer cells while shredding the rest of the workbook scores a success. Of the wider agent-benchmark literature, only OS-Harm [OS-Harm 2025]

measures agent side effects at all, and it measures categories of harmful action, not whether a file survived intact. The dimension along which #22044 failed is the one dimension no benchmark reports.

Three responses to this are natural, and each addresses a different problem than the one above. The first is to write a better prompt or skill and instruct the agent to be careful with the file. But guidance cannot make carelessness impossible; a guarantee has to live somewhere the agent cannot talk its way past, and the only such place is the runtime. The 2025–26 wave of runtime enforcement for agents agrees: deterministic runtime policies beat prompt- and heuristic-level defenses by wide margins (Progent drove one attack-success rate from 39.9% to 0% [Progent 2025], and AgentSpec, IsolateGPT, and ACID-snapshot sandboxing report similar separations [AgentSpec 2025; IsolateGPT 2024]). Yet every system in that wave is format-blind: it can gate a filesystem write or a shell command, but none understands the artifact it is guarding, so none can answer *will this edit corrupt the workbook's semantics?* or even *can this runtime evaluate this file at all?* The second response is to build a better agent — SheetCopilot and SheetAgent chase higher task-success rates — but the corruption those systems do not measure is orthogonal to how good the planner is; a perfect planner on top of openpyxl still ships #22044. The third response is to keep a human in the loop, which is the closest prior work: Pista [Pista 2026], from a group including Gulwani, gives a person step-level oversight of a spreadsheet agent, with each step explained in natural language. Pista is synchronous, does not scale to unattended operation, never verifies that a step's explanation matches what the code actually did, and never checks that the file survived; its constraints are, in its authors' words, "solely system prompt instructions," with "programmatic controls" named as future work. This paper walks through the door Pista left open: a machine-checked enforcement boundary that is format-aware and runs unattended.

This paper makes four contributions. First, it characterizes and measures the fidelity gap: a three-tier account showing that across openpyxl and LibreOffice alike, byte-identity is destroyed and cached values are untrustworthy — 101,961 blanked, 90,448 drifted — so any correct integrity check must be semantic and recompute-aware, together with the demonstration that no current agent benchmark reports this (Section 2). Second, it presents xlq, a format-aware and self-honest enforcement boundary: every operation an agent may run unattended is a read that is side-effect-free by construction, taken over a privacy-safe structural census that lets the agent read a workbook's structure rather than its contents (999 bytes versus a 7.24 MB full-cell dump, a 7,246.5× reduction, on a workbook of roughly 100,000 formulas), and the runtime states its own per-file evaluability blind spots as three numbers rather than one — 522 of 522 catalog functions recognized, 505 locally evaluable, 17 policy-limited and returning the exact error literal Excel returns (Sections 3–4). Third, it validates the boundary's formula engine with a documentation-arbitrated differential oracle: 1,659 hand-authored cases across 492 functions, run against LibreOffice as a reference peer with Excel documentation as the arbiter, which earned the engine's trustworthiness rather than assuming it and surfaced real bugs in both engines — of 85 value-versus-value disagreements, 72 received a decidable Excel-arbitrated fault assignment, 24 of them against our own engine (Section 5). Fourth, it presents AXLE-bench, a benchmark whose fidelity axis measures the dimension the benchmarks above omit, reported alongside its instrument validation and adoption cost (Section 6). We are explicit about what we did not invent: receipts, transactions, and differential testing are established; our contribution is making the enforcement boundary *format-aware and self-honest* for a legacy artifact class, and validating a formula engine where no mechanized specification exists. Receipts descend from receiver-attested agent logs [Notarized Agents 2026], forward-secure MAC logging [Nitro 2025], Certificate Transparency [Certificate Transparency 2022], and PROV [Moreau et al. 2013]; transactional isolation from ACID-snapshot and compensating-transaction sandboxes [Fault-Tolerant Sandboxing 2025]; and differential testing from a mature lineage [Yang et al. 2011; Rigger and Su 2020; Park et al. 2021; SmartOracle 2026] whose spec-as-arbiter members all presuppose a mechanized specification that the spreadsheet domain lacks.

1.3 2. The Fidelity Gap

When an LLM agent edits a workbook, the edit is not written to a log beside the artifact — it *becomes* the artifact. The cells the agent produces are the cells the file's owner will sign off on, submit to an auditor, and be held accountable for [Panko 2008; Hermans and Murphy-Hill 2015]. This promotes a property that task-success evaluation never names into a first-class safety concern: *fidelity* — whether the parts of the file the agent did not intend to touch survive the operation unchanged. This section argues, from the

failure of the field's own precedents, that fidelity is at once routinely disturbed by the substrate agents use and systematically unmeasured by the benchmarks they are scored against. In doing so it establishes the measurement discipline the rest of the paper rests on: byte-identity and cached values are untrustworthy across every substrate, so any correct integrity check must be semantic and recompute-aware (claim C1).

The precedent failure has two halves. The first is the substrate. The de-facto library for programmatic `.xlsx` editing — the one Anthropic's own agent skill ships and the one implicated in a shipped-agent corruption report (issue #22044, `anthropics/claude-code`) — is `openpyxl`, and a load-then-save round-trip through it does not return the bytes it was handed. The second is evaluation. Every current LLM-spreadsheet benchmark scores an agent by comparing designated answer cells to a reference and stops there; none inspects whether the rest of the file was preserved. Neither half is a law of the domain — each is a convention that has gone unexamined because no one measured across it. We measured across it.

Method. We took a five-workbook corpus and re-saved each file through three substrates: `openpyxl` 3.1.5, LibreOffice 24.8.7.2 (`--convert-to xlsx`, headless), and the vendored IronCalc engine (master `e50ccea8`). Each output was compared to its input at two levels: an OOXML part census (part names, uncompressed sizes, and content CRC32s from the zip central directory) and a cell-level *recompute-aware* semantic diff (§3), which recomputes every formula in the vendored engine and reports a distinct `cached_value` change-kind when a cell's stored cache differs from its input while its live recomputed value does not. One provenance caveat governs the IronCalc row: the fixtures were authored by IronCalc itself, so the IronCalc row is a *best case* for that engine's self-round-trip; a companion run (§D2) that feeds it `openpyxl`- and LibreOffice-authored files shows it rewrites foreign files as freely as the others. IronCalc is not the hero of this table; it is the row that shows what "no change" looks like as a baseline.

Byte-identity is destroyed everywhere. The part-survival result — the "OOXML parts left byte-identical" column of Table 1, computed per part from the zip central-directory census (`results.json`, `D_preservation`) — is first-order: re-saving rewrites essentially the whole package regardless of tool. LibreOffice rewrote *every* part of every fixture — not one part survived byte-identical. `openpyxl` rewrote all but the static theme part. It also dropped `xl/sharedStrings.xml` — but this is normalization, not loss: `openpyxl` re-inlines the strings into the sheet XML (verified: 170 shared-reference cells became 170 inline-string cells with every text value present), so the information is conserved while the byte layout is not. The consequence for tooling is decisive: because ~100% of content-bearing OOXML parts are rewritten by both mainstream substrates, a byte-level or CRC-level diff reports "everything changed" on a re-save that changed no user-visible value, and is therefore useless as an integrity oracle. A check that can separate a benign re-serialization from a value-altering edit cannot operate on bytes.

Three tiers, measured separately. Once byte-diff is discarded, the recompute-aware diff resolves the transformations into three distinct tiers, which must not be collapsed into a single word like "damage" (Table 1):

Substrate (version)	OOXML parts left byte-identical	Cached formula values affected	Character
<code>openpyxl</code> 3.1.5	theme only	101,961 blanked to <code><v/></code>	recoverab
LibreOffice 24.8.7.2	none	90,448 rewritten at 15 vs. 17 significant digits	below the
IronCalc master <code>e50ccea8</code>	all (self-authored)	0	best-case

Table 1. Re-save fidelity across substrates, five-workbook corpus (`results.json`, `D`).

T2 (semantics-preserving normalization). The shared-string re-inlining above is lossless; its only real effect is to defeat byte-provenance, already covered by the byte-diff result.

T2.5 (recoverable, but breaks cache-trusting consumers). `openpyxl` blanks the cached result of **every** formula it round-trips — **101,961** `<v/>` cells across the corpus (`results.json`, `D`; e.g. 442 in `branch-consolidation.xlsx`, 99,800 concentrated in the ~100k-formula `perf-large.xlsx`, but present in all five files). Because the formula text survives, the values are recoverable by recomputation; but any downstream consumer that reads the cache rather than recomputing — a viewer, an importer, a second agent, a `git diff`-style comparator — now sees empty cells where numbers were. This is the

concrete, measured motivation for a recompute-aware diff and its `cached_value` change-kind (§3): the same instrument that flags these 101,961 cells also confirms, by recomputation, that none of them represents a live semantic change.

T3 (cosmetic sub-precision drift). LibreOffice writes formula caches at 15 rather than 17 significant digits — **90,448** cells (results.json, D; almost all in `perf-large.xlsx`). This is a legitimate serialization choice, not a defect. The drift is below the round-trip precision of a double and is recomputed on open, so the live semantics are unchanged. Its only consequence is, again, that it defeats naive byte- and cache-level provenance. That both a value-erasing transformation (T2.5) and a benign precision choice (T3) surface through the *same* `cached_value` change-kind, and are told apart only by recomputation, is itself the argument for making the diff semantic.

What is not measured here. The one tier we do *not* measure is the most severe: T1, genuine irreversible loss of charts, pivot tables, or VBA. Our fixtures contain none of these features (IronCalc cannot author them), so we make no measured T1 claim. We flag this as an external-validity limitation and point to the external evidence that it is real: `openpyxl` is documented to strip exactly such parts from production financial models (issue #22044). The tiers we *can* measure already suffice for C1; T1 raises the stakes rather than establishing them.

The evaluation half of the gap. None of this is caught by how spreadsheet agents are currently scored. SheetCopilot (NeurIPS 2023), SheetAgent (WWW 2025), SpreadsheetBench (NeurIPS 2024) and SpreadsheetBench 2 (2026), and Finch (2025) all grade against target cells and are silent on the untouched remainder; SheetCopilot's checker skips unflagged properties *by design*. The single agent benchmark that scores any side effect at all, OS-Harm (NeurIPS 2025), judges harm categories in the agent's actions, not fidelity of the file it leaves behind. An agent could blank all 101,961 caches, report "done," and score a clean pass on every one of them.

Claim C1. Across every substrate an agent might use, re-saving an `.xlsx` destroys byte-identity and rewrites or blanks the cached values on which byte- and cache-level integrity checks depend; distinguishing a benign re-serialization from a value-changing edit therefore requires a semantic, recompute-aware comparison, and no existing spreadsheet-agent benchmark measures this dimension. The rest of the paper is the boundary and the evaluation that follow from taking that claim seriously.

1.4 3. A Format-Aware Enforcement Boundary

The boundary is built on a single discipline: everything an agent may do unattended is a read, reads are side-effect-free by construction, and the runtime states plainly what it cannot evaluate rather than returning a confident answer it cannot stand behind. The design owes its credibility to that radical simplicity — there is nothing to trust about a read that provably cannot write, and nothing to be misled by in a value the runtime has flagged as unverifiable.

The claim staked here is narrow. Format-awareness is not itself new: data-loss-prevention systems parse file formats to decide what may leave a network, and schema-aware database sandboxes understand the tables they guard well enough to reject a write that violates a constraint. What did not exist is an enforcement boundary for an LLM agent that is aware of its target artifact's *format fidelity*. To our knowledge, `xlq` is the first LLM-agent enforcement boundary that is artifact-format-fidelity-aware. This is not a new enforcement paradigm — the 2024–2026 runtime-enforcement wave already established that a deterministic runtime beats prompt-level guidance [Progent 2025; AgentSpec 2025; IsolateGPT 2024] — but every system in that wave is format-blind: it decides whether an action is *permitted* without understanding whether the action *corrupts* the artifact, or whether the runtime can even *evaluate* the artifact it is guarding. `xlq` folds the format into the boundary. We did not invent receipts, transactions, or differential testing; the contribution of this section is the format-aware layer, and it rests on the fidelity gap of §2 and is made trustworthy by the coverage accounting of §4 and the oracle of §5.

1.4.1 Read-only by construction

xlq v0.1 exposes exactly three commands — `inspect`, `diff`, and `calc` — and none of them can write the target file. This is a structural property, not a runtime check: the command dispatcher routes each subcommand to a function that loads the workbook, computes a report, and returns a JSON value, and the tool's only output sink is stdout. `calc` recomputes every formula and compares the recomputed value against the value stored on disk, but it reports the discrepancy rather than persisting it — its module contract reads "Never writes." The mutating surface (`apply` and `calc --write`) is not disabled in the read-only tool; it is physically absent — a separate, still-unbuilt specification (below) — so an agent driving xlq today has no code path that reaches a file write. The three read commands are exercised as pure report-returning functions, and every source file carries 95% line coverage.

1.4.2 A privacy-safe structural census

xlq `inspect` produces a *census*: a structural summary that answers "what does this workbook need from an engine?" without answering "what is in it?" A census reports sheet dimensions and cell/formula tallies, per-literal counts of the errors already stored in the file, the call-site frequency of each Excel function, which OOXML container parts are present, and the file's SHA-256 — and, by specification, never a cell value, never a fragment of a formula body (no string literals, constants, or cell references), and never a full filesystem path (`file.name` is a basename). The invariant is regression-tested against a sentinel workbook whose cells carry the markers `SECRET_VALUE_XYZ` and `SECRET_FORMULA_LIT`: the test serializes the census in both normal and redacted mode and asserts that neither marker — nor the file's parent directory — ever appears in the output.

Taking the format seriously forces one distinction a format-blind summarizer would miss: Excel vocabulary versus user data. A called name that is neither a canonical Excel function nor known to the engine — a VBA/XLL user-defined function, an add-in call, or a `LAMBDA` invoked through a defined name — is treated as user data, because such names routinely encode deal terms and client identities (a call site in our tests is `DealMargin_AcmeCorp`). These names are reported under a separate `user_defined_calls` member, are redactable, and never leak into the function tallies; a regression test confirms that `DealMargin_AcmeCorp` is reported as a user-defined call, is absent from the function map, and disappears entirely under `--redact`.

The census is also what makes the artifact legible to an agent at a fraction of the token cost of dumping cells. On a workbook of roughly 100k formulas (`perf-large.xlsx`), the census was 999 bytes against a 7,239,303-byte naive full-cell JSON dump — a 7,246.5× reduction (`results.json`). This number is not comparable to prompt-side content-compression schemes such as SpreadsheetLLM [Dong et al. 2024], which compress cell *contents* to support understanding and question-answering: the census discards contents entirely and keeps only the structure a mutation-safety decision needs, so the two ratios measure different tasks and are not placed head to head.

1.4.3 Coverage honesty

The census carries a `coverage` object with a single `reliable` flag and, when it is false, the exact reason. We name this primitive *coverage honesty*: a runtime that reports its own per-artifact blind spots instead of presenting an unqualified value. `reliable` is false whenever the workbook uses a function the engine does not know, a function the engine recognizes but deliberately refuses to compute (reported with the exact Excel error literal it yields), a user-defined callable the engine cannot evaluate, or an engine-level feature gap. The flag is *per file*: it answers "can this runtime be trusted about *this workbook*?", not "does the engine support Excel in general" — the catalog-level accounting is the three-number taxonomy of §4.

Coverage honesty is the first of two operational discriminators — concrete things a format-aware boundary does that a format-blind one cannot. Consider a workbook that is mostly evaluable but calls one function the runtime will not compute — say `WEBSERVICE`, whose true value depends on a live HTTP fetch a hermetic engine deliberately does not perform. A format-blind guard has two options, both wrong: recompute and hand back a number (silently substituting Excel's offline `#VALUE!` for the live value, presented as if authoritative), or refuse the workbook whole. xlq `calc` does neither. It recomputes every cell it can, and returns `coverage.reliable: false` together with `policy_limited_functions: {"WEBSERVICE": "#VALUE!"}` —

a typed statement naming the one function whose stored value it cannot verify. The agent learns precisely which cells not to trust, and why, and can still rely on the rest. The same mechanism fires for a UDF-bearing workbook: `reliable` becomes false and the offending callable is named (redactably). A boundary that does not model the artifact's function catalog cannot make this distinction; it can only be uniformly credulous or uniformly refusing.

1.4.4 The semantic diff and the specified write path

The second discriminator is in `xlq diff`. Because every calculation substrate rewrites nearly the entire OOXML container on save — `openpyxl` 3.1.5 re-inlines shared strings and rewrites parts, and LibreOffice writes caches at lower precision (§2) — a byte-level diff of two workbooks is useless as an integrity oracle: it reports that almost everything changed. A task-success metric has the opposite blind spot: it checks only the answer cells, which a careful save leaves correct. `xlq diff` instead compares workbooks semantically, cell by cell, on canonical formula text and raw stored value, and it separates a change in a *formula* from a change in a formula's *cached result*. When two cells carry the same formula but different stored values, the change is classified `cached_value` and counted in its own bucket, apart from genuine edits. This surfaces exactly the failure both other views miss: `openpyxl` writes an empty `<v/>` for every formula cell it saves, blanking the cached result while leaving the formula intact, so any downstream consumer that trusts caches reads nothing. Across our corpus this blanking touched 101,961 cached values — 442 on `branch-consolidation.xlsx` alone (results.json). The `cached_value` change-kind exists because this failure first hid inside `diff` itself, which once reported "1 change" while those results silently vanished — a finding we return to in the discussion. Here the point is only that the classification is what lets a semantic, recompute-aware diff, and nothing byte-level or task-level, name the damage.

The write path completes the boundary but is deliberately its least-built part. It is specified ahead of implementation and ships in v0.2. An agent proposes a *typed patch* (`set_cell` / `set_formula`, with an explicit JSON-to-Excel type mapping so the number-versus-text and date-serial traps are handled, never inferred) carrying the base file hash. A `--dry-run` writes nothing and predicts the affected cells, any *new* formula error, and before/after values for caller-named watch cells, degrading its own coverage flag when a volatile or unsupported function lies in the affected dependency graph. A real apply writes an immutable revision file, atomically swaps the workbook, and appends a receipt to an append-only, hash-chained sidecar journal; each receipt binds the pre-image hash to the post-image hash, and an edit made outside the tool breaks the chain detectably and is recorded as an `external_edit` receipt that *adopts* the new hash rather than wedging the file.

None of these ingredients is new, and the paper claims none of them. Append-only hash-chained receipts descend from forward-secure logging [Nitro 2025], Certificate Transparency's Merkle-proof logs [Certificate Transparency 2022], and W3C PROV / PROV-AGENT provenance [Moreau et al. 2013]; agent-action receipts specifically were introduced by Notarized Agents [Notarized Agents 2026]. Transactional rollback for agents is likewise established — ACID-snapshot sandboxing with 100% rollback and semantic and compensating transactions [Fault-Tolerant Sandboxing 2025; Cordon 2025; SAFEFLOW 2025]. Our journal is a domain instantiation, not a cryptographic construction: a single-writer local sidecar whose threat model is file custody and out-of-band edit detection rather than receiver attestation, and whose differentiator over a generic transactional sandbox is that the dry-run's prediction is *semantic* — affected cells, new formula errors, watched magnitudes — not merely atomic. The contribution of this section is not receipts or transactions; it is folding the artifact's format into the boundary so that reads are provably non-destructive, structure is shared without contents, and the runtime declares — per file — what it cannot stand behind.

1.5 4. Coverage Honesty: Three Numbers, Not One

A formula runtime is conventionally described by one number: how many of Excel's functions it supports. The convention encodes a false dichotomy. "Supported" silently fuses two questions the owner of a workbook needs answered apart — does the runtime *recognize* this function at all, and can it *compute the function's value from the workbook's own data?* — and it has no vocabulary at all for the case where the correct answer is a specific documented error. Binary supported/unsupported is a reporting convention, not a property of

formula evaluation. Before instantiating any measurement, this section replaces the bit with a partition of the space, and only then reports the counts that fill it. The partition, not the counts, is the contribution: it is what lets a runtime state its own blind spots per artifact rather than round them away. We call the primitive **the three-number coverage taxonomy**.

The three numbers. Against a pinned catalog — the 522 worksheet functions in Microsoft's alphabetical function list, fetched 2026-07-02 (`benchmarks/excel-functions.txt`) — the vendored engine measured (`benchmarks/coverage.json`):

#	Measure
1	Recognized — the name parses and dispatches to defined behavior, never a parser unknown-name rejection
2	Locally evaluable — full semantics computed from the workbook's own data
3	Policy-limited — recognized and argument-validated, then the documented desktop-Excel refusal literal, because the

Table 2. The three-number coverage taxonomy (`coverage.json`).

The three numbers are exhaustive and disjoint over the recognized set: $505 + 17 = 522$. Their measurement procedures differ, and we state them so the counts are not mistaken for more than they are. *Recognized* (522) is directly probed: each catalog name is driven through `census::probe_support` (below) and classified by the error it raises. *Policy-limited* (17) is an *enumerated* set — the specific functions in Table 3, each hand-curated with the exact refusal literal it returns — not a probe residual. *Locally evaluable* (505) is then, by construction, the recognized catalog minus that enumeration: a function is counted evaluable when its dispatch path computes a value from workbook data rather than returning one of the 17 enumerated refusal literals. So 505 is a *residual of the enumeration*, not an independent per-function probe that each of the 505 computes a defined, non-erroring value; a recognized function that dispatched to a stub which happened to error for some other reason would still fall in this count. What 505 asserts is therefore evaluability-by-dispatch, and nothing about *correctness* of the value dispatched — number 2 is an evaluability claim, number 3 a claim about *why* the remainder cannot be evaluated (made function by function with the exact literal desktop Excel returns), and number 1 a name-resolution claim. Reporting any one of them alone — the near-universal practice — is what makes “coverage” a number people distrust.

Guard: locally evaluable is not correct. Number 2 is the honest evaluability claim, and nothing more. Evaluability is *coverage* — the engine will compute a defined value from workbook data — whereas *correctness*, whether that value matches Excel, is the oracle's burden and is measured separately in §5 (claim C3). The 505 must never be read as “505 correct.” A function is in the evaluable count the moment it computes rather than refuses; whether it computes the *right* answer is a question this section does not, and cannot, answer. Coverage honesty means keeping these two axes apart on purpose, because collapsing them is precisely how a runtime overstates what it knows.

Policy-limited was hiding three reasons. A theory review flagged that “policy-limited (17)” was itself a fused label, papering over three materially different situations behind one word. Splitting them is required by the same honesty the taxonomy exists to enforce. The 17 decompose into a *capability* limit, a *policy* refusal, and a *context* precondition — three reasons a reader would want to act on differently:

Reason class	n	Functions
Capability-limited	14	WEBSERVICE RTD STOCKHISTORY, DETECTLANGUAGE, TRANSLATE, COPILOT, IMAGE CUBEVALUE, CUBEMEMBER, CUBESET, CUBESETCOUNT, CUBERANKEDMEMB
Policy-blocked	2	CALL, REGISTER.ID
Context-precondition	1	GETPIVOTDATA

Table 3. The 17 policy-limited functions, decomposed by reason (`coverage.json`).

The distinction is not pedantic. Capability-limited functions return the literal an *offline desktop Excel* returns; the limit is that the datum is not local, and xlq's local-only stance (memo §16, "no network calls") guarantees it stays not-local. Policy-blocked functions are refused even where the capability exists: CALL and REGISTER.ID invoke native code, so #BLOCKED! is a *refusal* literal, not an offline one — Microsoft blocks them and so does xlq's "preserve, never execute" principle. GETPIVOTDATA is neither remote nor hazardous; it needs a workbook object — a materialized PivotTable — that the engine's model does not contain, so #REF! is a *missing-precondition* literal. Three literals, three reasons, one honest label each.

The CUBE carve-out — recognition stays measured, not asserted. The probe drives the same `census::probe_support` path xlq `inspect` uses: it sets `=NAME(1)` in a scratch model and, by default, reads `#NAME?` as *unrecognized*, since in Excel an unknown name errors before argument validation. That heuristic is exactly wrong for the seven CUBE functions, whose documented correct answer with no OLAP connection is `#NAME?`. Treating them normally would wrongly deflate the recognized count — the taxonomy would understate its own breadth. The carve-out probes those seven names with zero arguments instead: a genuinely recognized cube function fails arity validation first (`#ERROR!`, never `#NAME?`), while a name the engine truly does not know still answers `#NAME?`. Recognition for the CUBE family is therefore *measured* — not asserted by fiat, and not inflated by a convenient default. Coverage honesty cuts both ways: the carve-out exists to stop the probe from flattering *and* from underselling the engine.

The probe's soundness rests on one assumption we make explicit: that a single canonical call — `=FN(1)`, or zero arguments for the CUBE carve-out — separates a recognized name from an unknown one, i.e. that `#NAME?` is raised *only* for genuine non-recognition and not, for some function, as that function's own legitimate one-argument result. The CUBE family is the one case in the 522 where that default misfires, and the carve-out is exactly the correction for it; the claim that no *other* catalog function returns `#NAME?` on a recognized one-argument call is a property of Excel's error ordering (an unknown name errors before argument validation), which holds for the catalog as probed but is an assumption of the measurement rather than an exhaustively certified fact. We report 522/522 **recognized** as the output of this probe under that stated assumption, not as an oracle-independent truth.

Why this is a runtime property, not a spec sheet. The same probe runs per workbook. When a file uses a policy-limited name, xlq `inspect` and xlq `calc` report it in the census's `policy_limited_functions` bucket (name → literal) and set `coverage.reliable: false` — the stored values genuinely cannot be *verified* locally — while `unsupported_functions` stays empty, because "the engine does not know this name" would be a lie. The runtime declares its own per-artifact evaluability blind spot up front, rather than computing a guess and reporting success. That per-file self-report, not the aggregate count, is what the taxonomy buys.

The journey to the numbers. The plan predicted that most of the engine's function set would have to be written from scratch. Reality disconfirmed it, in the strongest direction: measuring the substrate before building it showed the pinned release, IronCalc 0.7.1, recognized 345/522 (66.1%), but upstream *master* had already added roughly 148 functions since that pin. Reading the diff — not writing code — moved recognition to 494/522 (94.6%), which redirected the effort from rebuilding to vendoring master and contributing upstream. Three residual local-patch names (AGGREGATE, ENCODEURL, HYPERLINK) brought it to 497. A final full-catalog milestone genuinely implemented eight more with local semantics (FILTERXML, EUROCONVERT, DBCS, JIS, BAHTTEXT, PHONETIC, GROUPBY, PIVOTBY), taking *locally evaluable* to 505, and recognized the last seventeen as policy-limited with Excel-exact refusal literals, closing *recognized* to 522/522 (`docs/COVERAGE.md`). The lesson the taxonomy records is the one the substrate taught: measure before building, and report what the measurement actually partitions.

This makes claim **C2** — a formula runtime can recognize the full Excel catalog with defined, documented behavior and report that coverage *honestly*, as a three-number partition with a per-function literal for every non-evaluable case, rather than as a single supported-count that conflates recognition, evaluability, and correctness. The reframing is the claim: coverage honesty is a property a runtime can hold and disclose per artifact, and the three-number coverage taxonomy is the shape that property takes.

1.6 5. Documentation-Arbitered Differential Validation

The census (§3) and the coverage report (§4) are only as trustworthy as the formula engine beneath them: a vendored engine that computes a wrong value will have the runtime faithfully report a wrong value, and a coverage-honest claim that a function is *locally evaluable* (§4) says nothing about whether the evaluation is *correct*. Correctness is a separate burden — contribution C3 — and it cannot be discharged by assumption. Nor can it be discharged the way a compiler or a SQL engine's can, because there is no mechanized specification of spreadsheet formula semantics to check against. ECMA-376 standardizes the OOXML container, not the numerical behavior of the functions inside it; the one normative formula specification, OpenFormula, governs ODF, not Excel's dialect; Excel itself is defined by informal vendor prose. So the engine was validated the way compilers were before verified reference semantics existed [Yang et al. 2011]: by differential testing. Each of 1,659 hand-authored cases across 492 functions was run through two independent engines — the vendored IronCalc (master, e50ccea8) and LibreOffice 24.8.7.2 headless — and every divergence was treated as a signal to investigate, not a verdict. Of the 1,659 cases, the two engines agreed on 1,273, disagreed on a value in 85, both raised an error in 196, and hit 105 cases LibreOffice does not implement (15 of which IronCalc also errored on); no case crashed either engine (`engine_error` = 0, `benchmarks/agreement.json`).

1.6.1 The disagreement matrix (the result)

The scientific object is not the agreement count but the 85 cases in which both engines returned a value and the values differed. Each was adjudicated by hand against Excel's documented behavior, using the three-way verdict discipline of [Liu et al. 2024] — a divergence is a *bug*, an *ambiguity*, or *undecidable*, never auto-blamed on one side (`benchmarks/triage-analysis.md`):

Excel-arbitrated verdict	Count
IronCalc wrong	24
LibreOffice wrong	41
Both wrong	7
Spec-ambiguous	13
Undecidable here	0

Table 4. Excel-arbitrated triage of the 85 value-vs-value disagreements (*trriage-analysis.md*).

Seventy-two of the 85 disagreements (85%) received a decidable, documentation-grounded fault assignment; the remaining 13 were recorded as spec-ambiguous — transcendental-precision residuals, tail p-values, version-gated LAMBDA/ISOMITTED behavior — rather than forced to a verdict, and none was left undecidable-yet-blamed. Two properties of this matrix matter more than any single cell. First, it indicts the engine the runtime actually ships **24 times**: the harness is not rigged to flatter IronCalc, because a rigged harness does not fault its own engine two dozen times. Second, on the 18 core-semantics cases with zero ambiguity (TRIM, CONVERT, ROW, SUMPRODUCT, POWER, the A-suffixed aggregates, the type/coercion family) the split is nearly even — 8 IronCalc-wrong to 10 LibreOffice-wrong — while on the 21 financial cases it is lopsided the other way, 13 LibreOffice-wrong to 3 IronCalc-wrong, dominated by LibreOffice day-count and duration errors. The two engines fail in *different directions*, which is precisely why comparing them was informative.

1.6.2 Bugs in both engines

The verdicts resolve to concrete, filable defects. **IronCalc**: `CONVERT(68,"F","C")` returned 19.65 where Excel returns 20; `ROW` over a range returned a scalar instead of the array Excel spills; `SUMPRODUCT` did not coerce boolean arrays; `MAXA/MINA/STDEVA` ignored text and boolean cells that Excel counts; `TRIM` did not collapse internal whitespace runs; `SECOND` truncated sub-second times; `PRICE` at basis 3 returned exact par. **LibreOffice, deviating from Excel**: `POWER(0,0)` returned 1 where Excel returns `#NUM!` — and IronCalc *matched* Excel's `#NUM!`; a four-function cluster (`COUNT`, `ISNUMBER`, `TYPE`, `CONCATENATE`) treated boolean cells as plain numbers; four Treasury-bill functions used a wrong day count (reimplementing Excel's documented

formula in Python gave 184 days where LibreOffice used 181, and IronCalc matched the documented 184); `DURATION`/`MDURATION` diverged from documented Macaulay duration, which IronCalc reproduced to 1e-9. The 196 both-error cases carried signal too, though we triaged them less exhaustively than the 85 value disagreements and say so. Across the full population the error codes matched in 58 and mismatched in 138 (`agreement.json`); of that 196 we hand-adjudicated a 40-case sample for *which* engine's code matched Excel, and 30 of the 40 mismatched — in nearly every mismatch IronCalc returned Excel's exact error class (`#NUM!` for domain violations, `#N/A` for unit mismatch) while LibreOffice collapsed it to `#VALUE!`. Only the sample was adjudicated against Excel, so this is a directional indication that IronCalc mirrors Excel's error taxonomy, not a full triage of the 196; the asymmetry with the exhaustively-triaged value disagreements is a real gap in the error-class evidence, flagged rather than smoothed over.

1.6.3 Concordance is not accuracy

Only beneath the matrix does the aggregate concordance belong, and only with a disavowal. On the 1,311 cases whose outputs were numerically comparable, the two engines matched on 1,273 — **97.1%** (935 exact, 338 within tolerance); across all 1,358 cases in which both returned a value, 93.7% (benchmarks/`agreement.json`). The comparison policy that produces these counts is explicit and stored per case, because it is what decides the size of the disagreement set. "Numerically comparable" names the 1,311 of the 1,358 both-value cases in which *both* sides are numeric; the other 47 pair a number on one side with text or an error literal on the other, admit no numeric distance, and are excluded from the 97.1% denominator (they fall wholly inside the 85 disagreements). A comparable pair is scored *agree* when the two values are bit-identical (935 cases) or fall within the oracle's floating-point closeness test (338 cases); the per-case exact/within-tolerance/mismatch label and the tolerance itself are recorded in `agreement.json` rather than left to the reader to guess. The 38 comparable pairs outside tolerance plus the 47 non-comparable pairs are exactly the 85 value-vs-value disagreements ($38 + 47 = 85$; $1,273 \text{ agree} + 38 = 1,311$; $1,311 + 47 = 1,358$), so every denominator in this paragraph is reconstructable from the artifact. Concordance is *necessary but not sufficient* for correctness. Two engines that read the same ECMA-376 and the same vendor prose can converge on the same wrong behavior, so agreement bounds the *disagreement* count from below, not the *error* count from above. LibreOffice is a reference engine, not ground truth; this paper never claims "IronCalc is 97.1% correct." The only correctness statements it makes are the 72 arbitrated per-case verdicts above, each traceable to Excel documentation.

1.6.4 How fault was assigned, and how far it can be trusted

The arbitration protocol was deliberately weak where it could not be strong, and explicit where it could. The reference of record was Microsoft's published per-function documentation (the same office.com function-reference pages, captured 2026-07-02, that pin the 522-name catalog), read against the ECMA-376 text where it speaks. Because ECMA-376 standardizes the container and not the numerical semantics of the functions, a fixed precedence resolved conflicts: a *published closed-form formula* (day-count conventions, bond pricing, Macaulay duration) outranks vendor *prose*, and vendor prose outranks ECMA-376 silence; where none of the three decides the case, it was recorded spec-ambiguous rather than forced to a verdict. Where a published formula fixed the answer, that formula was reimplemented in Python and used to compute the expected value; `DURATION`, `MDURATION`, `PRICE` (basis 3), all four T-bill functions, `PRICEMAT`, and `AMORDEGRC` are high-confidence for this reason. Medium-confidence items (`PERCENTRANK` truncate-versus-round; `SKEW.P`/`SYD`/`CRITBINOM` boundaries) are flagged in the artifact for reviewer spot-check.

Two properties of this protocol bound how far it can be trusted, and we state them rather than let the verdict counts imply more rigor than exists. First, adjudication was performed by hand, by the authors — who also ship one of the two engines under test — without a second blind adjudicator or an inter-rater agreement figure; the 24 verdicts against our own engine show the harness is not tuned to flatter IronCalc, but a single adjudicator's 41 LibreOffice-wrong calls carry the weaker guarantee, and the per-case reasoning is published in `triage-analysis.md` precisely so a second reader can contest any of them. Second, and most consequentially, the largest fault asymmetry — 13 LibreOffice-wrong to 3 IronCalc-wrong on the financial functions — rests entirely on those author-written Python reimplementations, which are a single in-house third implementation of the day-count and pricing formulas, not validated against an independent financial

library or a licensed Excel. Their agreement with IronCalc (which reproduced the documented 184-day count and the Macaulay duration to 1e-9) is corroborating but not independent, so the financial asymmetry should be read as *high-confidence-but-single-source*; cross-checking those formulas against an external reference is the cleanest strengthening of this row and is future work. The oracle claims no more than this: a raw disagreement is an *inconsistency*, not a verdict — the distinction that [Pham et al. 2019] found differential pipelines routinely elide — and because the arbiter is external documentation applied by hand, the oracle's soundness never rests on trusting either engine under test, though it does rest, for the financial cases, on the one Python implementation named here.

1.6.5 What is, and is not, new here

None of the *method* is new. Generating inputs in a well-defined subset and cross-voting independent implementations is the differential-testing playbook [Yang et al. 2011]; arbitrating the divergences against a prose standard is exactly what CSmith does against the C standard — CSmith is itself prose-arbitrated. The contribution claimed for C3 is not a methodology but (a) the *domain*: spreadsheet formula engines, which unlike compilers [Yang et al. 2011], SQL engines [Rigger and Su 2020], and JavaScript engines [Park et al. 2021; SmartOracle 2026] have no mechanized or normative specification to build a stronger oracle from; (b) an explicit, artifact-backed arbitration protocol for that spec-less setting; and (c) the concrete bugs it surfaced in both engines. This is exactly where the strongest oracle-construction work parts company: PQS synthesizes a reference evaluator [Rigger and Su 2020], JEST treats a mechanized specification as an (N+1)th implementation [Park et al. 2021], and SmartOracle triages engine disagreements by *querying* the ECMAScript specification [SmartOracle 2026] — each affords an oracle stronger than cross-voting *because a normative spec exists*. In the spreadsheet domain none does, so the honest oracle is a weaker one, and it is reported with its two blind spots named. First, the 1,659 cases were hand-authored — roughly 3.4 per function — not coverage-guided fuzzing, so 85 disagreements is a *lower bound* on divergence; a generative campaign would very likely surface more. Second, the strongest available arbiter was not used as a peer: Excel-the-binary is an executable oracle through COM or Office-JS, but no licensed Excel environment was available on the Linux host, so Excel entered only as documentation. Adding Excel as a third differential engine — turning documentation arbitration into a third vote — is the single clearest strengthening, and it is stated as future work rather than claimed here.

This is what earns the runtime's honesty (§9, load-bearing assumption 1): the engine's outputs are trusted because they are differential-validated with documentation arbitration, not because IronCalc is assumed correct, and where the oracle is silent or both engines are unvalidated, the coverage report (§4) flags the cell rather than the runtime reporting a number it cannot defend.

1.7 6. AXLE-bench: Measuring the Omitted Dimension

Prior agent benchmarks score whether the task cells came out right; none reports whether the file survived. Of the spreadsheet-agent evaluations surveyed in §8, SheetCopilot, SheetAgent, SpreadsheetBench, and Finch grade only answer-cell match [Li et al. 2023; SheetAgent 2025; SpreadsheetBench 2024; Finch 2025], and SheetCopilot's checker skips unflagged properties by design; among agent benchmarks generally, only OS-Harm measures side effects at all, and it measures process-level harm rather than file fidelity [OS-Harm 2025]. AXLE-bench (Agent-safe eXcel Layer Evaluation) is a consolidated suite that makes the fidelity gap of §2 a first-class measurement axis, placed beside the four dimensions on which a spreadsheet tool is ordinarily judged. Following OS-Harm's taxonomy-before-benchmark discipline, we fix the five axes first and then instantiate every tool against them symmetrically. Every number in this section traces to a JSON artifact in `benchmarks/` at commit 150fb66, and one meta-runner (`run_all.sh`) reproduces them on a single machine (Intel i7-8700K, 12 logical cores, Linux), skipping with a notice any sub-suite whose prerequisites are absent rather than failing.

1.7.1 The five axes

1. **Correctness — the oracle agreement.** Cross-engine value agreement between the vendored engine (ironcalc 0.7.1+e50ccea8) and LibreOffice 24.8.7.2 over 1,659 hand-authored cases across 492

functions. The two engines agree on **97.1% of value-producing cases (1,273/1,311)**, and on **93.7% (1,273/1,358)** once one-side-error rows are counted (`agreement.json`; the 1,311 numerically-comparable subset and the exact/within-tolerance rule that partitions it are defined in §5). This is *concordance*, not accuracy: two engines reading the same ECMA-376 and vendor prose can share a wrong behavior, so the scientific result is the disagreement triage of §5, not the percentage. With Excel documentation as arbiter, of the 85 value-level disagreements **24 were IronCalc-wrong and 41 LibreOffice-wrong** (7 both, 13 spec-ambiguous, 0 undecidable; `triage-analysis.md`) — the harness indicts its own engine 24 times, which is the anti-stacking evidence that it is not tuned to flatter the engine it ships. LibreOffice here is a reference peer, not ground truth.

2. **Fidelity — the omitted dimension.** Two sub-measures on re-save: OOXML part survival (drop/add/rewrite per part) and cached-value integrity via `xlq diff`, over a five-workbook corpus totalling 101,961 formula cells (`results.json`, §D/D2). A single `openpyxl` re-save **blanks all 101,961 cached formula values** (99,800 of them in the 100k-cell fixture alone) — every formula cell reads empty in a cache-trusting consumer until recalculation, the tier-2.5 loss that motivates a recompute-aware, semantic integrity check. LibreOffice instead drifts **90,448 cached values** by re-serializing 17-significant-digit stored floats at 15 digits — last-ulp reformatting, recomputed on open, not corruption, but enough to defeat any byte-level provenance check. The vendored engine records **0 cached-value drifts** on all five fixtures, but that row is provenance-biased: the fixtures were authored by its own writer, and section D2 shows that on `openpyxl`- and LibreOffice-authored inputs it rewrites every part and re-emits its own metadata parts, the same re-serialize-the-world behavior as everyone else. Across every substrate, byte-identity is destroyed and caches are untrustworthy, which is precisely why fidelity must be checked semantically. The fixtures contain no VBA, charts, or pivot tables (the engine cannot author them), so the irreversible-loss failure of [claude-code#22044] is untested here, not disproven.
3. **Efficiency — median-of-3 warm wall time.** On the ~99,800-formula fixture (`results.json`, §A): `xlq calc` (load + full recalculation + a stored-vs-recomputed audit + SHA-256 + JSON report) takes **1.264 s**; the engine's isolated load is **0.599 s**; `libreoffice --convert-to xlsx` takes **1.494 s**; and `openpyxl load_workbook` takes **0.875 s, with recalculation n/a — it has no formula engine.**
4. **Agent-ergonomics — the census token ratio.** Bytes of the structural census versus a naive full-cell JSON dump, as a token proxy (`results.json`, §C): on the 100k-cell book the census is **999 B versus 7,239,303 B, a 7,246.5× reduction**; on the small branch-consolidation fixture, 1,800 B versus 51,107 B, **28.4×**. Inspect latency is 7–20 ms on the small fixtures and 0.908 s on the 100k-cell book, and every command emits JSON on stdout. (The census is structure-only for mutation safety; it is not comparable head-to-head with lossy content-compression ratios reported for understanding tasks [Dong et al. 2024].)
5. **Catalog — the three-number coverage.** Over Microsoft's 522-name worksheet-function catalog (`coverage.json`): **522/522 recognized (100%), 505 locally evaluable, 17 policy-limited.** The three numbers are the honest accounting developed in §4 — “locally evaluable” is coverage, not correctness, and the 17 decompose into 14 capability-limited (needing network or live data), 2 policy-blocked (XLM CALL/REGISTER.ID, refused for security), and 1 context-precondition (GETPIVOTDATA), each returning the exact error literal Excel returns.

1.7.2 Instrument validation as a co-headline

AXLE-bench's measuring instruments are themselves fallible, so we report their limits beside their outputs rather than in a footnote. The correctness axis has no live Excel oracle: we lack a licensed Excel on this Linux machine, so per-case verdicts rest on documented semantics, and adding Excel as a third differential peer is the clearest strengthening (§5). The 1,659 cases are hand-authored at roughly 3.4 per function — modest input diversity — which makes the disagreement rate a **lower bound**, and 36 functions received no oracle signal at all. The fidelity axis compares zip central-directory part names, uncompressed sizes, and content CRC32s, a byte-level basis that is why cosmetic drift (LibreOffice's 90,448) has to be separated from consumer-breaking loss (`openpyxl`'s 101,961) by hand. The sharpest instrument-validation point is that the

fidelity tool caught a bug in itself: an early `xlq diff` reported one change on an `openpyxl` re-save while 442 formula results had silently vanished — the exact damage the boundary exists to catch, hiding inside the measuring tool — which is why the `cached_value` change-kind exists (developed in §9). We report these blind spots because the property the suite is built to measure is a tool's willingness to report its own.

1.7.3 Adoption cost and known floors

Reproduction cost is a design metric, reported as OS-Harm reports its dollar cost [OS-Harm 2025]. The full suite runs on one commodity machine under a fixed protocol — one untimed warmup plus three warm runs, median wall clock — and sub-20% deltas should be read as noise, not signal. The corpus is deliberately declared a floor: five synthetic fixtures, all authored by the engine under test (a best case for its own round-trip), with stronger inputs meant to slot into the same harness. We state these floors so that a reader can see exactly where a harder fixture or a live-Excel peer would move the numbers.

1.7.4 The comparison matrix

Every cell below is exactly one of: a number traceable to a **benchmarks/** artifact (marked [A] agreement, [R] results, [C] coverage); **n/a (cannot)** — the tool cannot perform the operation by design, with its documentation cited; or **not-run (cite)** — a documented-capability row we did not benchmark here because no runnable, comparable setup exists in this environment, carrying a citation instead of a fabricated number. Star counts are as of 2026-07-02 via the GitHub API; capability citations resolve through `docs/BASELINE.md`.

Tool	Correctness (1,659-case oracle)
xlq 0.1.0 / ironcalc-master (RUN HERE)	97.1% agreement where both engines value (1,273/1,311); 93.7% inc
openpyxl 3.1.5 (RUN HERE)	n/a (cannot) — "never evaluates formula" (docs; BASELINE §1)
LibreOffice 24.8 headless (RUN HERE)	Reference peer; triage attributes 41 disagreements to LO (POWER
excel-mcp-server (~3,978) — NOT-RUN-HERE	not-run — openpyxl underneath → cannot evaluate (pyproject.toml
OfficeCLI (~8,299) — NOT-RUN-HERE	not-run — vendor claims "350+ functions"; no agreement data exists

Table 5. The AXLE-bench comparison matrix. [A]/[R]/[C] mark the source artifact.

1.7.5 Methodology caveats, stated straight

The matrix reports where `xlq` loses on the same artifacts. **openpyxl parses faster than xlq calculates** — 0.875 s versus 1.264 s — because `xlq`'s number buys a full recalculation of 99,800 formulas plus a stored-vs-recomputed audit that `openpyxl` cannot perform at any speed; the like-for-like parse comparison is the engine's isolated load at 0.599 s, against which `openpyxl`'s 0.875 s is entirely respectable. **LibreOffice's 1.494 s is not a bound on its load-plus-recalc cost: --convert-to** spawns a process, loads, and writes, but Calc does not recalculate `xlsx` on load by default (BASELINE §4), so recalculation is likely absent from that number; an honest engine-vs-engine figure would need a UNO-scripted hard recalc, which this harness does not run. And the clean preservation row belongs to the engine only on files its own writer authored; section D2 shows it re-serializes foreign files like every other tool.

1.7.6 The claim this suite supports

Two columns are blank for every other row, run-here or not-run, open source or eight-thousand-star. **No other tool in the matrix wraps agent writes in preview → apply → receipt → revision** — not `openpyxl`, not LibreOffice, not `excel-mcp-server`, not OfficeCLI (verified against each tool's own documentation, `docs/BASELINE.md`, "Gaps"). We did not invent the primitives that would fill it: change receipts have prior art [Notarized Agents 2026; Nitro 2025; Certificate Transparency 2022; Moreau et al. 2013], and transactional rollback is established [Fault-Tolerant Sandboxing 2025]; `xlq` v0.1 earns its cell honestly, read-only today with the write path spec'd rather than claimed. **And no other tool reports its own blind spots** — every `xlq inspect/calc` output carries `coverage.reliable` plus the named unsupported and policy-limited functions *for that workbook*, while every other row in the matrix answers confidently or

not at all, and none of them tells you when it is guessing. That gap — not raw speed — is the dimension AXLE-bench exists to measure, and to keep honest.

1.8 7. Results

This section is a digest, not a re-run: it states each axis's headline once and points to the derivation, rather than re-tabulating §2–§6. The only table it repeats is the fidelity-and-cost consolidation (Table 6), which no earlier section presents in one place; the coverage census (Table 2) and the disagreement matrix (Table 4) are cited, not reprinted. All figures come from `benchmarks/*.json` at commit 150fb66, on a single machine (Intel i7-8700K, 12 logical cores, Linux) with the vendored engine `ironcalc 0.7.1+e50ccea8`. The five axes map to the two contribution claims — format-awareness (fidelity and coverage-honesty) and documentation-arbitered differential validation (the oracle); none of the instruments is novel in isolation, and the result is what they measure when pointed at spreadsheet artifacts.

Coverage honesty (Table 2, §4). Against the 522 worksheet functions in Microsoft's alphabetical list (`coverage.json`), the runtime recognized every one; 505 are locally evaluable and 17 policy-limited (14 capability-limited, 2 policy-blocked, 1 context-precondition), each returning Excel's exact error literal rather than a fabricated value. "Locally evaluable" is a coverage claim (C2), never "505 correct" — the correctness burden is the oracle's, below. Coverage rose from 345/522 (66.1%) at the pinned 0.7.1 release to 494 (94.6%) by vendoring engine master and to 505 after Tier-I implementation work; that journey, and why the honest unit is three numbers rather than one, is §4.

Differential oracle (Table 4, §5). The headline is the disagreement matrix, not the concordance rate. Of 1,659 hand-authored cases across 492 functions (~3.4/function, so the disagreement count is a lower bound; no case crashed the engine, `engine_error` = 0), 85 were value-vs-value disagreements; Excel-documentation arbitration assigned decidable fault in 72 — 24 against our own engine, 41 against LibreOffice, 7 both-wrong, 13 spec-ambiguous. The split ran both ways within the zero-ambiguity strata (core-semantics 8-IC vs 10-LO; financial 3-IC vs 13-LO). The harness indicting IronCalc 24 times is why we lead with faults rather than the concordance figure, which is demoted to context: 1,273/1,311 = 97.1% of value-producing cases (935 exact, 338 within tolerance; §5 defines the subset and tolerance), 93.7% once one-side-error rows are counted. LibreOffice is a *reference* peer, not ground truth — "the engines agree on 97.1% of cases" is defensible; "IronCalc is 97.1% correct" is not.

Fidelity and cost across substrates. Every substrate that re-saves a workbook destroys byte-identity and leaves cached formula values untrustworthy, but the mechanisms differ enough that a single "damage" number would misrepresent them; the measurements are reported by tier.

Substrate (version)	Operation timed on <code>perf-large.xlsx</code> (~100k formulas)	Median (s)	Cached-value beha
xlq (this work)	load + full recalc + stored-vs-recomputed audit	1.264	read-only by const
IronCalc (master e50ccea8)	load only	0.599	near-byte-identical
openpyxl 3.1.5	load only (no formula engine; calc n/a)	0.875	blanks all cached r
LibreOffice 24.8.7.2	<code>--convert-to:</code> spawn + load + save (not isolated recalc)	1.494	precision drift, 15

Table 6. Fidelity and recompute cost across substrates (`benchmarks/results.json`; median of 3 warm runs; cell counts corpus-wide). The four timings measure different operations and are not directly comparable — only xlq's includes a full recalc and a stored-vs-recomputed audit, and it completed in 1.264 s. LibreOffice's 1.494 s is not a fair comparator in either direction: it includes process-spawn overhead but, by default, no `xlsx` recalculation (§6), so we draw no "faster" conclusion from it; the like-for-like parse figure is the engine's isolated load at 0.599 s. T1 (irreversible loss of charts, pivots, or VBA) was **not** measured on this corpus because the IronCalc-authored fixtures contain none; it is stated as a limitation and supported only by external evidence (openpyxl stripping such structure on real financial models, issue #22044). The measured tiers make the design case directly: since openpyxl blanks 101,961 cached values (442 of them on `branch-consolidation.xlsx`) and LibreOffice drifts 90,448, a byte-diff is useless as an integrity oracle

and the check must be semantic and recompute-aware — which is what `xlq`'s `diff` and the `cached_value` change-kind are.

Agent ergonomics. The structural census encoded `perf-large.xlsx` in 999 B against 7,239,303 B for a naive full-cell JSON dump — a $7,246.5\times$ reduction — and `branch-consolidation.xlsx` in 1,800 B against 51,107 B ($28.4\times$) (`benchmarks/results.json`). This is not comparable head-to-head with SpreadsheetLLM's $25\times$ [Dong et al. 2024]: that task is lossy content compression for understanding, whereas the census is structure-only for mutation safety.

Test coverage. `xlq` carried 67 tests (up from 57), with every `src/*.rs` file at 95% line coverage under `cargo-llvm-cov`; the vendored engine's full suite ran 2,187+ tests green (~940 of them the engine's own function tests).

Read together, the trust axes (coverage honesty and the fault-assigning oracle) and the mechanism axes (fidelity, recompute cost, and census compression) support the same narrow claim: an enforcement boundary that is aware of the spreadsheet format can report where it cannot evaluate, prove where the substrate corrupts, and do so at a cost competitive with tools that verify nothing — while the oracle's 24 self-indictments keep the trust claims from resting on the runtime's own word.

1.9 8. Related Work: What We Did Not Invent

We place related work after the mechanism so that the reader meets the boundary before the crowded literature it draws on, and so the following disclaimers read as precision rather than defense. We did not invent receipts, transactional sandboxing, or differential testing. The claimed contribution is narrower and, we argue, defensible: making the enforcement boundary *format-aware* — carrying fidelity and per-artifact coverage-honesty — and validating a formula engine by *documentation-arbitered* differential testing in a domain that has no mechanized specification. This section credits the lines we build on and states exactly where our core sits inside them.

Runtime enforcement for LLM agents. A 2024–2026 wave established that deterministic runtime enforcement outperforms prompt- and heuristic-level defenses. Progent [Progent 2025] reports 0% attack success under deterministic policies against 39.9% for prompt-based mitigation; AgentSpec [AgentSpec 2025] compiles runtime rules from a DSL; MiniScope [MiniScope 2025] derives least-privilege OAuth scopes by integer programming; IsolateGPT [IsolateGPT 2024] enforces process isolation between tools; provenance-graph monitors [Agent-Sentry 2025] track cross-tool data flow. A transactional sub-line makes writes reversible: ACID-snapshot sandboxing [Fault-Tolerant Sandboxing 2025] reports 100% filesystem rollback, and semantic- and compensating-transaction systems [Cordon 2025; SAFEFLOW 2025] extend this to multi-step actions. We take the shared empirical finding — that a runtime is the only place a guarantee can live — as settled, and adopt it. The shared blind spot is that every one of these systems is format-agnostic: none understands the artifact it guards, and so none can answer “will this edit corrupt the file's semantics?” or “can this runtime even evaluate this artifact?” Our transactional layer is not novel for being atomic; its differentiator is *dry-run with semantic prediction* — the affected cells, newly introduced formula errors, and changed watch values reported before a write — which requires the format-aware engine that the enforcement wave does not carry.

Receipts and provenance. Cryptographic action receipts and tamper-evident logs are established. Notarized Agents [Notarized Agents 2026] issues receiver-attested receipts for agent actions; forward-secure MAC logging [Nitro 2025], Certificate Transparency's Merkle-proof audit logs [Certificate Transparency 2022], W3C PROV and PROV-AGENT [Moreau et al. 2013], and BlockAudit supply the provenance ancestry. Our receipt journal is a domain instantiation of these ideas, not a new cryptographic construction: a single-writer local sidecar with a hash chain and out-of-band edit-adoption semantics, targeting a different threat model — custody of a file that a human co-edits — rather than receiver attestation. We say so plainly, and note that in the present system the journal is specified rather than exercised at evaluation scale.

Differential testing. The oracle-construction menu is mature: cross-vote generation in a well-defined subset [Yang et al. 2011], metamorphic self-differential compilation [Le et al. 2014], a tool-built reference evaluator used as the arbiter [Rigger and Su 2020], optimizer metamorphic relations [Rigger and Su 2020],

a mechanized specification as the (N+1)-th implementation [Park et al. 2021], and LLM-mediated triage against a normative spec [SmartOracle 2026], with Liu et al. [Liu et al. 2024] contributing the three-way verdict vocabulary (bug / inconsistency / underspecified) that our triage adopts. Positioned against this: while cross-engine comparisons of spreadsheet numerical accuracy against certified reference values exist [Almiron et al. 2010; McCullough 2008], and vendors privately benchmark formula compatibility [HyperFormula 3.1.0], no prior work has applied generative differential testing to spreadsheet formula engines with an automated, documentation-grounded triage of disagreements; existing spec-as-arbiter techniques [Park et al. 2021; Liu et al. 2024; SmartOracle 2026] presuppose a mechanized or normative specification that the spreadsheet domain lacks. Excel's formula semantics live in informal vendor prose plus ECMA-376 text, so our arbiter is documentation, not a normative grammar. Our instantiation contributes that documentation-arbitered triage over 1,659 curated cases (§5); we are explicit that the case set is hand-authored, not coverage-guided fuzzing, and that scaling to a generative campaign is future work.

Spreadsheet research and LLM spreadsheet agents. The motivation bedrock is replicated error-pervasiveness work: Panko [Panko 2008] found errors in 94% of 88 audited sheets at a 5.2% cell error rate, and Hermans and Murphy-Hill [Hermans and Murphy-Hill 2015] found Excel errors in 24% of formula-bearing Enron sheets, with 75% of sheets using only the top-15 functions — independent support for a workload-weighted view of coverage. The EUSES corpus [Fisher and Rothermel 2005] and ExceLint's dependence-vector fingerprints [Barowy et al. 2018] are the classical ancestors of our structural census. On the agent side, SheetCopilot [Li et al. 2023], SheetAgent [SheetAgent 2025], SpreadsheetBench [SpreadsheetBench 2024], its successor (~594 cell modifications per task, best model 34.89% [SpreadsheetBench 2026]), and Finch (38.4% on real finance workflows [Finch 2025]) all score task-answer cells by exact match. The load-bearing gap, verified across these systems, is that none measures file corruption, preservation of untouched content, or side effects; SheetCopilot's checker skips unflagged properties by design, and only OS-Harm [OS-Harm 2025] measures agent side effects at all — and not file fidelity. On representation, our census occupies a third design point distinct from model-side encoders [Wang et al. 2021] and prompt-side lossy compression for understanding tasks [Dong et al. 2024, 25×/96% token reduction]: a tool-side, structure-only summary for mutation safety (7,246.5× on the large fixture). The compression numbers are not comparable head-to-head — the tasks differ.

Pista — the closest prior. Pista [Pista 2026], with Gulwani among its authors, is the nearest system: step-level human oversight of a spreadsheet agent, with model-generated explanations of each action. Its guarantees are, by its authors' account, prompt-only, its trust accrues through synchronous human participation, and it neither verifies that an explanation matches the executed snippet nor measures whether the file survived intact. This is a real and different problem — interactive accountability — and we do not diminish it. But Pista's own Limitations name our contribution as their future work: its constraints were "solely system prompt instructions" and want "programmable controls." We walk through that door. Where Pista promotes "semantic diff" as its named primitive, we promote *coverage-honesty* and the *three-number coverage taxonomy*; where Pista requires a human in the loop, our boundary is machine-checked — a differential-validated engine, per-file coverage-honesty, and receipts — and format-aware, so it operates unattended and reports what Pista's synchronous auditor cannot: whether the artifact the user is accountable for is still intact.

1.10 9. Discussion

1.10.1 Is a boundary that only reads, and only *predicts* writes, worth building?

We answer with the failure we found in our own tool. On a re-saved copy of `branch-consolidation.xlsx`, `xlq diff` reported *one change* while 442 formula results had silently vanished — the exact damage the boundary exists to catch, hiding one abstraction below where we were looking. An `openpyxl` round-trip had blanked every `<v/>` cache in the file (part of the 101,961 blanked across the five-workbook corpus; results.json, tier T2.5), yet the diff, comparing formatted display strings, saw only the single cell whose *rendered* text moved and pronounced the file intact. The tool built to make silent corruption visible was itself rendering corruption invisible. It was not code review that caught this but surface verification — running the tool end to end and reading its output against the file — the same discipline the boundary is supposed to impose on an agent. The fix replaced formatted-string comparison with recompute-aware raw-value comparison and added an explicit `cached_value` change-kind, so a blanked cache is now a first-class reported change rather

than a rounding artifact below the display. The bug that most validated the thesis was in our own hands, and it makes the case better than any argument we could stage: a boundary that only reads is worth building precisely because the read itself is where the fidelity signal lives, and getting the read wrong is easy.

This mirrors the load-bearing weakness of every differential tool in this lineage — CSmith's guarantees bottom out in the analyses it runs on its own output [Yang et al. 2011], PQS trusts the reference evaluator it wrote by hand [Rigger and Su 2020]. Our diff is no different: its correctness is an assumption about our own code, and the honest thing is to report the one time that assumption failed rather than the many times it held.

1.10.2 When the substrate moved faster than the plan

Three predictions in the ledger were recorded, run, and disconfirmed; we report them because the disconfirmations are the strongest gradient in the work.

Predicted rebuild; reality was read-the-diff. We expected the engine we built on to require most of the vision's early phases written from scratch. Instead, IronCalc *master* had already added roughly 148 functions since the pinned release. The honest move was to vendor master and contribute upstream, not to rebuild: local evaluability rose from 66.1% (345/522) to 94.6% (494/522) by reading the upstream diff, not by writing calc code (coverage.json history). The substrate outran the plan; the lesson was to measure the ground before pouring foundations.

Predicted a clean "100%"; reality was three numbers. A bare coverage percentage turned out to be dishonest in three distinct directions. The defensible statement is 522/522 catalog functions *recognized*, 505 *locally evaluable*, and 17 *policy-limited* — and even "17" hid three reasons a theory pass forced apart: 14 capability-limited (they need network or live data the runtime has no local access to, e.g. `WEBSERVICE`), 2 policy-blocked (`CALL/REGISTER.ID`, refused for security), and 1 context-precondition (`GETPIVOTDATA`, needs a workbook object we do not model). Forcing `WEBSERVICE` to fetch URLs would raise the percentage and violate the security thesis in the same edit. The taxonomy is more correct than the number it replaced.

Predicted the oracle would confirm the engine; reality was bugs in both. We expected the differential oracle to mostly ratify IronCalc. Of 1,659 hand-authored cases across 492 functions (agreement.json), 85 were value-versus-value disagreements, and Excel-documentation arbitration assigned decidable fault in 72 of them: 24 `IRONCALC_WRONG`, 41 `LIBREOFFICE_WRONG`, 7 `BOTH_WRONG`, 13 `SPEC_AMBIGUOUS` (triage-analysis.md). The harness indicted its *own* engine 24 times — `CONVERT(68,"F","C")` returned 19.65 where Excel gives 20; a `ROW`-over-range array was dropped — which is the anti-stacking evidence that it was not rigged to flatter IronCalc. It also caught the reference engine deviating from Excel (`POWER(0,0)→1` where Excel returns `#NUM!` and IronCalc matches Excel). The check we built as a gate became a contribution: a validation instrument that finds real bugs on both sides of the comparison.

1.10.3 Load-bearing assumptions, stated plainly so they can be challenged

We name three assumptions a reviewer should press on rather than bury them.

First, *engine trust is differential-earned, not assumed.* The vendored engine's outputs are trustworthy because they were validated against LibreOffice with Excel documentation as arbiter — LibreOffice is a *reference*, never ground truth, and we never claim IronCalc is "97.1% correct"; concordance is necessary, not sufficient. Two engines reading the same ECMA-376 prose can share a wrong behavior, which is why the headline is the disagreement matrix and not the agreement rate. Where the oracle is silent or both engines are unvalidated, coverage honesty is what flags the blind spot. The assumption is bounded: 1,659 cases at ~3.4 per function is modest input diversity, so the disagreement count is a *lower bound*, and the absence of a licensed Excel binary on our Linux machine means Excel-the-executable never sat in the loop as a third peer — the clearest strengthening available, and future work.

Second, *the census's privacy invariant holds.* The census reports structure, not content, and treats even user-defined function names as user data. This is substantiated by a regression test asserting that a sentinel cell value never appears in census output — but it remains an invariant, and a reviewer should probe it as one rather than take the test as a proof of the general case.

Third, *the prediction ledger is retrospective and says so*. For the systems work the ledger was reconstructed; a prediction is claimed only where it was genuinely recorded before the run that tested it. We state this in the open because a ledger that quietly back-dated its own successes would corrupt exactly the honesty the paper is about.

1.10.4 Two primitives we hope outlast this system

Beyond `x1q`, two ideas here are domain-general. The first is *coverage honesty*: a runtime that reports its own per-artifact evaluability blind spots instead of failing silently or claiming a capability it lacks. Any tool that interprets a rich artifact — a query planner, a linter, a model-context extractor — can declare, per input, what it can and cannot faithfully evaluate. The second is the *three-number coverage taxonomy*: recognized / locally evaluable / policy-limited, with the policy reason attached, replacing the binary supported/unsupported convention that hides security and capability decisions inside a single percentage. Neither primitive depends on spreadsheets; both are the residue of being forced, by our own disconfirmations, to stop rounding honesty down to one number.

We are explicit about what these primitives are *not*. We did not invent receipts (cf. Notarized Agents [Notarized Agents 2026], forward-secure MAC logging [Nitro 2025], Certificate Transparency [Certificate Transparency 2022], W3C PROV [Moreau et al. 2013]), transactional rollback (ACID-snapshot sandboxing [Fault-Tolerant Sandboxing 2025]), or differential testing (CSmith [Yang et al. 2011], PQS [Rigger and Su 2020], JEST [Park et al. 2021], SmartOracle [SmartOracle 2026]). The claimed novelty is narrower and, we hope, more durable: making an enforcement boundary *format-aware* — fidelity plus coverage honesty — and validating a formula engine by *documentation-arbitered* differential testing where no mechanized specification exists.

1.10.5 The door Pista left open

Pista [Pista 2026] made spreadsheet agents overseeable through synchronous human auditing of model-generated step explanations, and named its own successor problem: its guarantees were “solely system prompt instructions” and wanted “programmatic controls.” That is the door we walk through. Prompt-level constraints cannot verify that an explanation matches the executed action, cannot certify that the file survived intact, and do not scale to unattended operation. A machine-checked boundary can supply what a prompt cannot: read-only by construction, a structure-only census, and an engine whose correctness is differentially earned and whose blind spots it declares — with typed, dry-run, receipt-backed writes specified to complete it (§10, v0.2). We are precise about the boundary between built and specified: what this paper *implements and measures* is the read-only half — a boundary that cannot write, a census that cannot leak, an engine validated against a documentation arbiter. The typed-write mediation that would let an agent edit *through* the boundary is specified, not yet built, so the enforcement delivered today is enforcement-by-absence of a write path rather than enforcement-by-gating of one. We do not claim to have closed the problem — the census invariant, the retrospective ledger, the missing Excel oracle, and above all the unbuilt write path are all open on our side. But the read-only enforcement Pista's prompt-level constraints could not provide is implemented and measured here, and the most honest evidence that it was worth building is the 442 results our own tool once let vanish, and now does not.

1.11 10. Limitations

Each limitation below is derived from a specific mechanism in the system rather than offered as generic hedging; where a limitation bounds a headline number, we name the number and the direction it bounds.

Curated cases, not a generative-scale campaign. The oracle's 1,659 cases are hand-authored at roughly 3.4 per function — a validation of the vendored engine, not a coverage-guided fuzzing campaign. Because input diversity is modest, the 85 disagreements are a *lower bound* on divergence; a generative campaign would very likely surface more, and 36 functions received no oracle signal at all. Scaling the case generation, and adding Excel-the-binary as a third differential peer (unavailable on our Linux host), are the two clearest strengthenings — stated as future work, not claimed here.

Single-machine benchmarks. Every AXLE-bench timing comes from one commodity machine (Intel i7-8700K, 12 logical cores, Linux), median-of-3 warm, so sub-20% deltas are noise, not signal. The four substrate timings measure different operations (only xlq's includes a full recalc and audit) and are not head-to-head; no cross-hardware, cold-start, or scaling distribution is reported.

Retrospective prediction ledger. For the systems work, the disconfirmation ledger (§9) was reconstructed after the fact; a prediction is claimed only where it was genuinely recorded before the run that tested it. We flag its retrospective status rather than present it as a pre-registered protocol, because a ledger that quietly back-dated its successes would corrupt exactly the honesty the paper is built on.

The census drops phonetic runs. The structural census — and the engine's import path beneath it — does not preserve furigana `rPh` phonetic runs from `sharedStrings`. `PHONETIC` therefore evaluates against base text with its runs dropped (returning the cell's own text unchanged), and LibreOffice does not implement `PHONETIC`, so those cases carry no oracle signal (they fall in `lo_unsupported`). Any workbook whose semantics or fidelity depend on phonetic runs is outside what the census and the fidelity check currently model.

The write path is specified, not implemented. Typed patches, `--dry-run` semantic prediction, atomic revision swap, and the append-only hash-chained receipt journal are specified ahead of implementation and ship in v0.2; xlq v0.1 is read-only by construction. Every write-path guarantee in §3 is therefore a design claim exercised only at specification scale, not measured at evaluation scale. Relatedly, the fidelity corpus is five synthetic fixtures authored by the engine under test — a best case for its own round-trip — and contains no charts, pivot tables, or VBA, so the most severe tier of loss (T1) is supported only by external evidence [claude-code#22044], not measured here.

1.12 11. Conclusion

This paper set out to give an LLM agent a boundary it cannot cross without leaving evidence, for an artifact class whose owner is accountable for every cell. Four things were learned in building it.

First, the fidelity gap is real and unmeasured: across every substrate tested, byte-identity is destroyed and cached values are untrustworthy — blanked by `openpyxl` (101,961 cells), drifted by LibreOffice (90,448 cells) — so a correct integrity check must be semantic and recompute-aware, not a byte diff. This is the dimension current agent benchmarks omit.

Second, the honest unit of coverage is not a percentage. We had predicted that "100% coverage" was a clean target; it is truthful only as three numbers — 522 functions recognized, 505 locally evaluable, 17 returning the exact error literal Excel returns. We discarded the bare-100% claim for this taxonomy.

Third, trust in a formula engine can be earned rather than assumed, and doing so is itself a contribution. We had predicted a from-scratch engine rebuild; the upstream diff disconfirmed it — vendored master had already added ~148 functions, lifting coverage from 66.1% to 94.6% by reading the diff, not writing code — so we discarded the rebuild and contributed upstream instead. The differential oracle, arbitrated by documentation where no mechanized specification exists, then indicted both engines, including our own 24 of 85 times. The finding that most validated the thesis was in our own tool: diff reported one change while 442 formula results silently vanished, until we added the `cached_value` change-kind.

Fourth, an evaluation can measure fidelity: AXLE-bench adds the axis the field's benchmarks leave out.

We claim no new cryptography, no new transaction model, and no new differential-testing technique — receipts [Notarized Agents 2026; Nitro 2025; Certificate Transparency 2022; Moreau et al. 2013], transactional rollback [Fault-Tolerant Sandboxing 2025], and cross-engine differential testing [Yang et al. 2011; Rigger and Su 2020; Park et al. 2021; SmartOracle 2026] are all established. Our contribution is to make the enforcement boundary format-aware and self-honest for a legacy artifact class, and to validate an engine where no mechanized specification exists. We name two primitives we hope outlast this system: *coverage honesty* — a runtime that reports its own per-artifact blind spots — and the *three-number coverage taxonomy* that replaces the binary supported/unsupported convention.

Two lines of successor work follow directly from what the measurements exposed. Our own fidelity axis showed that xlg's engine rewrites foreign files as fully as the substrates it replaces; surgical-patch v0.2 will edit only the changed cells, closing that foreign-file fidelity gap. And the typed-patch, dry-run, and hash-chained receipt layer is specified but not yet implemented; building it — and adding Excel itself as a third differential peer, the clearest strengthening of the oracle — are the next steps. In taking them we answer the door Pista left open [Pista 2026]: the machine-checked enforcement its authors deferred to future work.

1.13 References

Runtime enforcement for LLM agents

- **[Progent 2025]** Progent: Programmable Privilege Control for LLM Agents. 2025. (Reports 0% attack success under deterministic runtime policies vs. 39.9% for prompt-based mitigation.)
- **[AgentSpec 2025]** AgentSpec: Runtime Enforcement of LLM-Agent Safety via a Domain-Specific Rule Language. 2025.
- **[MiniScope 2025]** MiniScope: Least-Privilege OAuth Scope Derivation for Agents via Integer Programming. 2025.
- **[IsolateGPT 2024]** IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. 2024.
- **[Agent-Sentry 2025]** Agent-Sentry: Provenance-Graph Monitoring of Cross-Tool Agent Data Flow. 2025.
- **[Fault-Tolerant Sandboxing 2025]** Fault-Tolerant Sandboxing for LLM Agents: ACID Snapshots with 100% Filesystem Rollback. 2025.
- **[Cordon 2025]** Cordon: Semantic Transactions for Multi-Step Agent Actions. 2025.
- **[SAFEFLOW 2025]** SAFEFLOW: Compensating-Transaction Safety for Agent Workflows. 2025.

Receipts and provenance

- **[Notarized Agents 2026]** Notarized Agents: Receiver-Attested Cryptographic Receipts for Agent Actions. 2026.
- **[Nitro 2025]** Forward-Secure MAC Logging for Tamper-Evident Audit Trails. ACM CCS, 2025.
- **[Certificate Transparency 2022]** SoK: Certificate Transparency and Merkle-Proof Audit Logs. IEEE Symposium on Security and Privacy, 2022.
- **[Moreau et al. 2013]** L. Moreau et al. The PROV Data Model (W3C PROV) and PROV-AGENT. W3C Recommendation, 2013.

Differential testing

- **[Yang et al. 2011]** X. Yang, Y. Chen, E. Eide, J. Regehr. Finding and Understanding Bugs in C Compilers (CSmith). PLDI, 2011.
- **[Le et al. 2014]** V. Le, M. Afshari, Z. Su. Compiler Validation via Equivalence Modulo Inputs (EMI). PLDI, 2014.
- **[Rigger and Su 2020]** M. Rigger, Z. Su. Testing Database Engines via Pivoted Query Synthesis (PQS/SQLancer) and Non-Optimizing Reference Engine Construction (NoREC). OSDI / ESEC-FSE, 2020.
- **[Park et al. 2021]** J. Park et al. JEST: N+1-Version Differential Testing of JavaScript Engines with a Mechanized Specification. ICSE, 2021.
- **[SmartOracle 2026]** SmartOracle: LLM-Mediated Triage of JavaScript-Engine Disagreements Against the ECMAScript Specification. 2026.
- **[Liu et al. 2024]** Liu et al. Specification-Based Differential Testing of SQL Engines with a Three-Way Verdict (bug / inconsistency / underspecified). 2024.
- **[Pham et al. 2019]** H. V. Pham et al. CRADLE: Cross-Backend Validation to Detect and Localize Bugs in Deep-Learning Libraries. ICSE, 2019.
- **[Almiron et al. 2010]** M. Almiron et al. On the Numerical Accuracy of Spreadsheets. Journal of Statistical Software, 2010.
- **[McCullough 2008]** B. D. McCullough. Assessing the Reliability of Statistical Software in Spreadsheets. 2008.

- **[HyperFormula 3.1.0]** HyperFormula 3.1.0 Formula-Compatibility Benchmark Suite (vendor, unpublished). Handsontable, 2024.

Spreadsheet research and LLM spreadsheet agents

- **[Panko 2008]** R. R. Panko. What We Know About Spreadsheet Errors. 2008. (Errors in 94% of 88 audited sheets; 5.2% cell error rate.)
- **[Hermans and Murphy-Hill 2015]** F. Hermans, E. Murphy-Hill. Enron's Spreadsheets and Related Emails: A Dataset and Analysis. ICSE, 2015. (Excel errors in 24% of formula-bearing sheets; 75% of sheets use only the top-15 functions.)
- **[Fisher and Rothermel 2005]** M. Fisher, G. Rothermel. The EUSES Spreadsheet Corpus. 2005.
- **[Barowy et al. 2018]** D. W. Barowy, E. D. Berger, B. Zorn. ExceLint: Automatically Finding Spreadsheet Formula Errors. OOPSLA, 2018.
- **[Li et al. 2023]** H. Li et al. SheetCopilot: Bringing Software Productivity to the Next Level Through Large Language Models. NeurIPS, 2023.
- **[SheetAgent 2025]** SheetAgent: A Generalist Agent for Spreadsheet Reasoning and Manipulation. WWW, 2025.
- **[SpreadsheetBench 2024]** SpreadsheetBench: Towards Challenging Real-World Spreadsheet Manipulation. NeurIPS Datasets & Benchmarks, 2024.
- **[SpreadsheetBench 2026]** SpreadsheetBench 2: Successor Benchmark (~594 cell modifications per task; best model 34.89%). 2026.
- **[Finch 2025]** Finch: Evaluating LLM Agents on Real-World Finance Spreadsheet Workflows (38.4%). 2025.
- **[OS-Harm 2025]** OS-Harm: Measuring Harmful Side Effects of Computer-Use Agents. NeurIPS, 2025.
- **[Wang et al. 2021]** Z. Wang et al. TUTA: Tree-Based Transformers for Generally Structured Table Pre-Training. KDD, 2021.
- **[Dong et al. 2024]** H. Dong et al. SpreadsheetLLM: Encoding Spreadsheets for Large Language Models (25×/96% token reduction). 2024.
- **[Pista 2026]** Pista: Step-Level Human Oversight of Spreadsheet Agents. 2026.

Standards and artifacts

- **[claude-code#22044]** Anthropic. claude-code issue #22044: xlsx agent skill corrupts financial models (charts, pivots, VBA) via openpyxl. GitHub, 2025.
- **ECMA-376 / ISO-IEC 29500.** Office Open XML File Formats. Ecma International / ISO. (Standardizes the OOXML container, not formula numerical semantics.)
- **OpenFormula.** OpenDocument Formula Specification (ODF). OASIS. (Normative for ODF, not Excel's dialect.)